Part A: Probability & Conceptual Foundation (Theory)

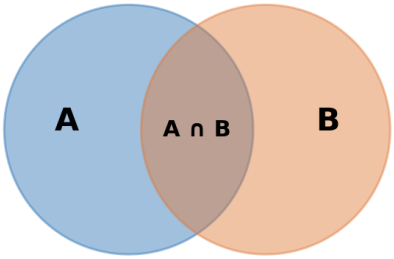
Conditional Probability · Bayes' Theorem · Naive Bayes · KNN · SVM

1. What is Conditional Probability?

Conditional probability is the probability that event **A** occurs **given that event B has already occurred**. Knowing B shrinks the sample space to B, so we measure how much of B also belongs to A.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

Fig 1: P(A|B) = share of B that is also A

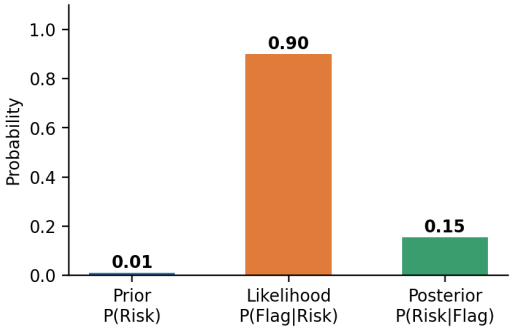


2. Explain Bayes' Theorem and its importance in classification problems.

Bayes' Theorem updates our belief about a class after seeing evidence: **posterior = likelihood × prior / evidence**. In classification, X is the feature vector and C_k a class; we compute $P(C_k|X)$ for every class and pick the largest, which gives the minimum-error (Bayes optimal) decision. It is the foundation of Naive Bayes and of probabilistic models in general.

$$P(C_k \mid X) = \frac{P(X \mid C_k)\,P(C_k)}{P(X)}$$

Fig 2: Bayes update (example)



*Example: if only 1% of transactions are risky and a detector flags 90% of risky and 5% of safe ones, a flagged transaction is risky with probability only **0.15** – the prior matters.*

3. What assumptions does the Naive Bayes Classifier make?

• **Conditional independence:** given the class, all features are independent of each other (this is the "naive" part).
• **Equal importance:** every feature contributes equally to the outcome.
• **Assumed feature distribution:** e.g. Gaussian for continuous data, Multinomial/Bernoulli for counts or binary data.
• **Representative training data** for estimating priors and likelihoods.

$$P(x_1, \ldots, x_n \mid C_k) = \prod_{i=1}^{n} P(x_i \mid C_k)$$

$$\hat{y} = \arg\max_k P(C_k) \prod_{i=1}^{n} P(x_i \mid C_k)$$

The assumption is rarely true in real data, yet Naive Bayes often still performs well and is fast and works with little data.

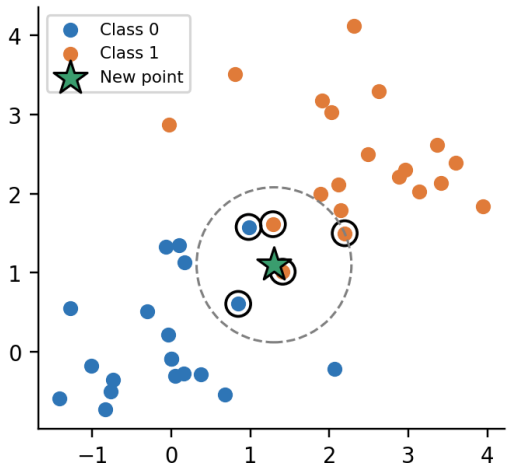
4. Explain the working principle of:

a) K-Nearest Neighbors (KNN)

KNN is a lazy, distance-based learner: it stores the training data and, for a new point, finds the **k closest** training points (usually by Euclidean distance) and assigns the **majority class** (or the average value for regression). Small k gives flexible but noisy boundaries; large k gives smoother ones. Feature scaling is important.

$$d(x, x') = \sqrt{\sum_{i=1}^{n} (x_i - x'_i)^2}$$

Fig 3: KNN (k=5) → predicts Class 1



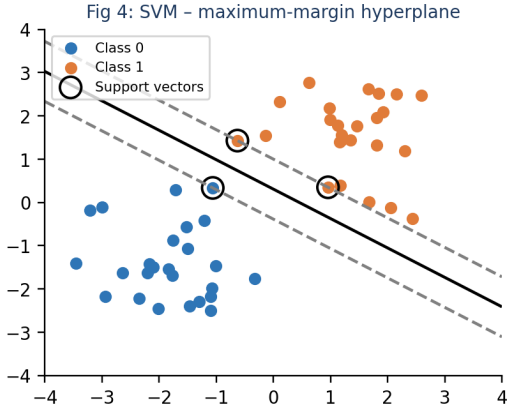
b) Support Vector Machine (SVM)

SVM finds the **hyperplane that separates the classes with the maximum margin**, i.e. the widest gap between the closest points of each class. Those closest points are the **support vectors** and alone define the boundary. Soft-margin SVM (parameter C) tolerates some errors, and **kernels** (RBF, polynomial) let it handle non-linear data.

$$f(x) = \text{sign}(w \cdot x + b), \quad \text{margin} = \frac{2}{\|w\|}$$

$$\min_{w,\,b} \frac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1$$

Fig 4: SVM – maximum-margin hyperplane



5. Compare distance-based, probabilistic, and margin-based classifiers.

They differ in *how* they decide a class: KNN uses **proximity**, Naive Bayes uses **probability**, and SVM uses a **maximum-margin boundary**.

Aspect	Distance-based (KNN)	Probabilistic (Naive Bayes)	Margin-based (SVM)
Core idea	Similar points are close together	Pick class with highest posterior probability	Maximise gap between classes
Training	None (lazy, stores data)	Very fast: count / estimate statistics	Slower: solves an optimisation problem
Prediction speed	Slow on large data	Very fast	Fast (uses support vectors)
Output	Class vote	Class probabilities	Class (distance to boundary)
Key assumption	Distance is meaningful; needs scaling	Features independent given class	Classes separable with margin / suitable kernel
Strengths	Simple, non-linear boundaries	Works with small data, text and spam tasks	Strong in high dimensions, robust to overfit
Weaknesses	Sensitive to noise, scale, large k	Independence assumption is unrealistic	Hard to tune, less interpretable

Fig 5: Decision boundaries on the same dataset

